

Appendix

1 ASP Formal Semantics

Answer Set Programming is a relational formalism where variables in the input language are generally replaced by constant symbols in a *grounding* step resulting in a propositional (ground) representation. We recall the formal semantics of ASP for ground programs. A *rule* r has the form

$$H \leftarrow B^+, \text{not } B^-$$

where H is a head element (an atom, a choice, or a penalty), B^+ is a set of positive body atoms, and B^- is a set of default-negated atoms. A *program* Π is a finite set of ground rules.

An *interpretation* I is a set of ground atoms. I satisfies a standard rule r (denoted $I \models r$) if $B^+ \subseteq I$ and $B^- \cap I = \emptyset$ implies $H \in I$. I satisfies a program Π ($I \models \Pi$) if $I \models r$ for all $r \in \Pi$.

The *reduct* Π^I of Π w.r.t. an interpretation I is the positive program defined as:

$$\Pi^I = \{H \leftarrow B^+ \mid (H \leftarrow B^+, \text{not } B^-) \in \Pi, B^- \cap I = \emptyset\}.$$

An interpretation A is an *answer set* (stable model) of Π if $A \models \Pi$ and no proper subset $I \subset A$ satisfies $I \models \Pi^A$. Answer sets embody the *closed-world assumption*: an atom is false unless explicitly derived by a well-founded support.

Aggregates and Choices. A choice rule $t_1 \leq \{A\} \leq t_2$ allows any subset $S \subseteq A$ to be included in I , provided its cardinality falls within the bounds $[t_1, t_2]$. Aggregates (e.g., #SUM, #COUNT) evaluate a mathematical function over a multiset of elements extracted from the current interpretation I , comparing the result against a threshold k .

Optimization. For programs with weak constraints (penalties $[w@l]$ with weight w at priority level l), a cost is computed for each level by summing the weights of all violated penalties. Optimal answer sets $AS^*(\Pi)$ are those that minimize this sequence of costs lexicographically.

Program Tightness. The positive dependency graph of a program Π is a directed graph where the vertices are the ground atoms, and there is an edge from A to B if there is a rule $r \in \Pi$ such that A is in the head of r and $B \in B^+$. A program Π is *tight* if its positive dependency graph contains no directed cycles. For tight programs, Fages' Theorem [?] guarantees that the

stable models coincide exactly with the classical models of the program's Clark Completion.

The semantics of SDL, defined via its translation into ASP, are detailed in the previous work [?].

2 Formal Semantics of Constraint Programming.

To formally define the semantics of the CP target and highlight its differences from ASP, we introduce the underlying mathematical framework of Constraint Satisfaction Problems (CSPs).

Let $X = x_1, \dots, x_n$ be a finite set of *decision variables*. Each variable $x \in X$ is associated with a finite *domain* $D(x) \subset Z$, representing the set of admissible integer (or Boolean, where $\{false, true\} \equiv \{0, 1\}$) values.

An *assignment* (or valuation) ν is a mapping from a subset of variables $Y \subseteq X$ to values, such that $\nu(x) \in D(x)$ for all $x \in Y$. A *complete assignment* is defined on all variables in X . The application of ν to a variable or expression e is denoted by $\nu(e)$.

A *constraint* c over a scope $S(c) \subseteq X$ is a mathematical relation defining the allowed combinations of values for the variables in $S(c)$. Formally, $c \subseteq \prod_{x \in S(c)} D(x)$.

For a complete assignment ν , the satisfaction relation $\nu \models \cdot$ is defined recursively as follows:

- for a primitive constraint c , $\nu \models c$ if the projection of ν onto $S(c)$ belongs to c ;
- for a conjunction, $\nu \models c_1 \wedge c_2$ if $\nu \models c_1$ and $\nu \models c_2$;
- for a classical negation, $\nu \models \neg c$ if $\nu \not\models c$ (unlike ASP's NAF, this strictly implies that the assignment violates the mathematical relation c);
- for a global universal quantification over a finite index set K , $\nu \models \forall i \in K (c_i)$ if $\nu \models c_i$ for every $i \in K$;
- for a sum aggregate of the form $\sum_{i \in K} e_i \odot v$ (where $\odot \in \{<, \leq, \geq, >, =, \neq\}$), $\nu \models \sum_{i \in K} e_i \odot v$ if the arithmetic expression holds under ν . If e_i is a Boolean logical expression, it is coerced into an integer via an indicator function $I_{\nu \models e_i}$ (evaluating to 1 if true, 0 if false).

A *Constraint Satisfaction Problem (CSP)* is a tuple $\mathcal{P} = \langle X, D, \mathcal{C} \rangle$, where $\mathcal{C}$ is a finite set of constraints. A complete assignment ν is a *solution* to $\mathcal{P}$ (denoted $\nu \models \mathcal{P}$) if $\nu \models c$ for all $c \in \mathcal{C}$. Let $Sol(\mathcal{P})$ be the set of all solutions to $\mathcal{P}$.

A *Constraint Optimization Problem (COP)* extends a CSP with an objective function $f(\nu)$ mapping assignments to real numbers (or integers). For a minimization problem, a solution ν^* is an *optimal solution* if $\nu^* \in Sol(\mathcal{P})$ and no $\nu \in Sol(\mathcal{P})$ satisfies $f(\nu) < f(\nu^*)$. Let $Sol^*(\mathcal{P})$ be the set of optimal solutions.

3 Correctness Sketch

Let $\text{Sol}(S)$ denote the intended abstract semantic space of an SDL specification S . Taking the ASP mapping Π_S^{ASP} as the reference semantics, we provide evidence that the CP mapping $\mathcal{M}_S$ yields consistent interpretations. Specifically, for any assignment to the guessed records, we argue that an interpretation valid under the non-monotonic ASP semantics has a natural structural counterpart that satisfies the monotonic CP constraints (with scalarized optimization), up to projection on the records mentioned in **show** directives. The argument proceeds by structural induction on the SDL instructions.

1. **Guess:** Both mappings enumerate equivalent Cartesian domains. The total and finite nature of the CP decision variables natively covers the exact search space generated by the ASP choice rules.
2. **Define (non-recursive):** For each non-recursive **define** instruction, the mapping generates a biconditional constraint $R(\mathbf{x}) \leftrightarrow \bigvee_i B_i(\mathbf{x})$ across all defining rules. This construction directly implements Clark’s Completion: the forward direction ($\leftarrow$) encodes the supporting rules, while the backward direction ($\rightarrow$) enforces minimality, ruling out any truth assignment not justified by an explicit rule body. Since non-recursive instructions induce no positive cycles in the dependency graph, the program is by definition tight. By Fages’ Theorem [?], the models of Clark’s Completion coincide precisely with the stable models for tight programs. Therefore, the biconditional encoding natively produces the same derived facts as the ASP back-end, including correct handling of Negation as Failure within rule bodies, which is mapped to classical negation under the Domain Closure Assumption (see the *Deny* paragraph below).
3. **Define (recursive):** For non-tight programs containing positive cycles, Clark’s Completion alone is insufficient to characterise the stable models: classical logic admits *unfounded sets*, i.e., sets of atoms that mutually support each other without any grounded base case, and therefore evaluate to true under the completion despite having no well-founded derivation. The rank-based encoding rules these out by construction. For each positively recursive record R , the mapping introduces a companion integer array $\text{rank}_R : D_R \rightarrow \{0, \dots, |D_R|\}$ and posts the acyclicity implication:

$$R(\mathbf{x}) \rightarrow \bigvee_{\mathbf{y}} \left(B(\mathbf{x}, \mathbf{y}) \wedge \text{rank}_R[\mathbf{x}] > \text{rank}_R[\mathbf{y}] \right).$$

Suppose, for contradiction, that a non-empty unfounded set $U \subseteq D_R$ exists in a solution; that is, every atom $R(\mathbf{x})$ with $\mathbf{x} \in U$ is true, yet no atom in U has a supporting derivation whose body is grounded outside U . By the implication above, each $\mathbf{x} \in U$ must be supported by some $\mathbf{y} \in U$ with $\text{rank}_R[\mathbf{x}] > \text{rank}_R[\mathbf{y}]$. This induces a strictly decreasing chain within U . Since U is finite and the rank domain is bounded below by

0, no such infinite descent exists ; a contradiction. Therefore every true atom in R must ultimately be traceable to a base case outside any cycle, which aligns with the well-foundedness condition of stable model semantics. This argument corresponds to the classical level mapping characterisation [?] of unfounded sets [?], instantiated here over the finite CP domain; the connection to stable models for tight programs then follows by Fages’ Theorem [?]. Conversely, every stable model A yields a valid rank assignment by setting $rank_R[\mathbf{x}]$ to the derivation depth of $R(\mathbf{x})$ in A ; well-foundedness of stable models guarantees this satisfies the acyclicity implication.

4. **Deny:** Both mappings enforce the same integrity prohibitions. The critical subtlety lies in Negation as Failure: in ASP, $\neg_{\text{naf}} R(\mathbf{x})$ holds whenever $R(\mathbf{x})$ cannot be derived, reflecting the Closed World Assumption. In the CP mapping, this is rendered as $\neg \exists \mathbf{y} \in D_R : \varphi(\mathbf{x}, \mathbf{y})$. The soundness of this translation rests on the **Domain Closure Assumption (DCA)**: because every domain D_R generated by $\mathcal{M}(S)$ is strictly finite and explicitly bounded, the exhaustive enumeration of D_R is complete. Consequently, the failure to find a witness $\mathbf{y}$ satisfying φ within D_R is treated as consistent with classical falsity; there is no open-world extension in which $R(\mathbf{x})$ could become true. The DCA is guaranteed by construction: the compiler never generates unbounded or parametric domains, ensuring that classical negation over D_R faithfully captures the non-monotonic NAF semantics of the original SDL specification.
5. **Weak Constraints:** SDL soft denials are stratified into priority levels $1, \dots, k$, with higher levels taking lexicographic precedence. The CP mapping scalarizes this hierarchy by assigning a static multiplier μ_L to each level, defined inductively as:

$$\mu_1 = 1, \quad \mu_L = \mu_{L-1} \cdot (\text{MaxCost}_{L-1} + 1),$$

where

$$\text{MaxCost}_L = \sum_{c \in \mathcal{W}_L} \sum_{\mathbf{x} \in \text{Dom}(c)} C_c(\mathbf{x})$$

is the worst-case cumulative penalty at level L . We argue by induction on L that this scaling preserves the global lexicographic ordering. The base case is trivial: at level 1, $\mu_1 = 1$ and no lower level exists. For the inductive step, assume that for all levels $1, \dots, L-1$ the scaling correctly preserves lexicographic dominance. By construction:

$$\mu_L = \mu_{L-1} \cdot (\text{MaxCost}_{L-1} + 1) > \text{MaxCost}_{L-1}.$$

This means that a single unit of improvement at level L contributes μ_L to the scalar objective, which strictly exceeds the maximum total penalty achievable across all constraints at level $L-1$ combined. Moreover, by

unrolling the induction one can verify that

$$\mu_L = \prod_{j=1}^{L-1} (\text{MaxCost}_j + 1)$$

from which the global dominance follows directly:

$$\mu_L > \sum_{j=1}^{L-1} \mu_j \cdot \text{MaxCost}_j.$$

Dominance is therefore global and transitive across the entire priority hierarchy. Consequently, no accumulation of penalties at any combination of lower levels can compensate for a higher-level violation, and the minimization of:

$$\text{total_penalty} = \sum_{L=1}^k \mu_L \cdot p_L$$

is mathematically isomorphic to lexicographic minimization over $(p_k, \dots, p_1)$. The induction closes at level k , establishing that the scalarized CP objective faithfully reproduces the priority structure of ASP weak constraints across all levels.